



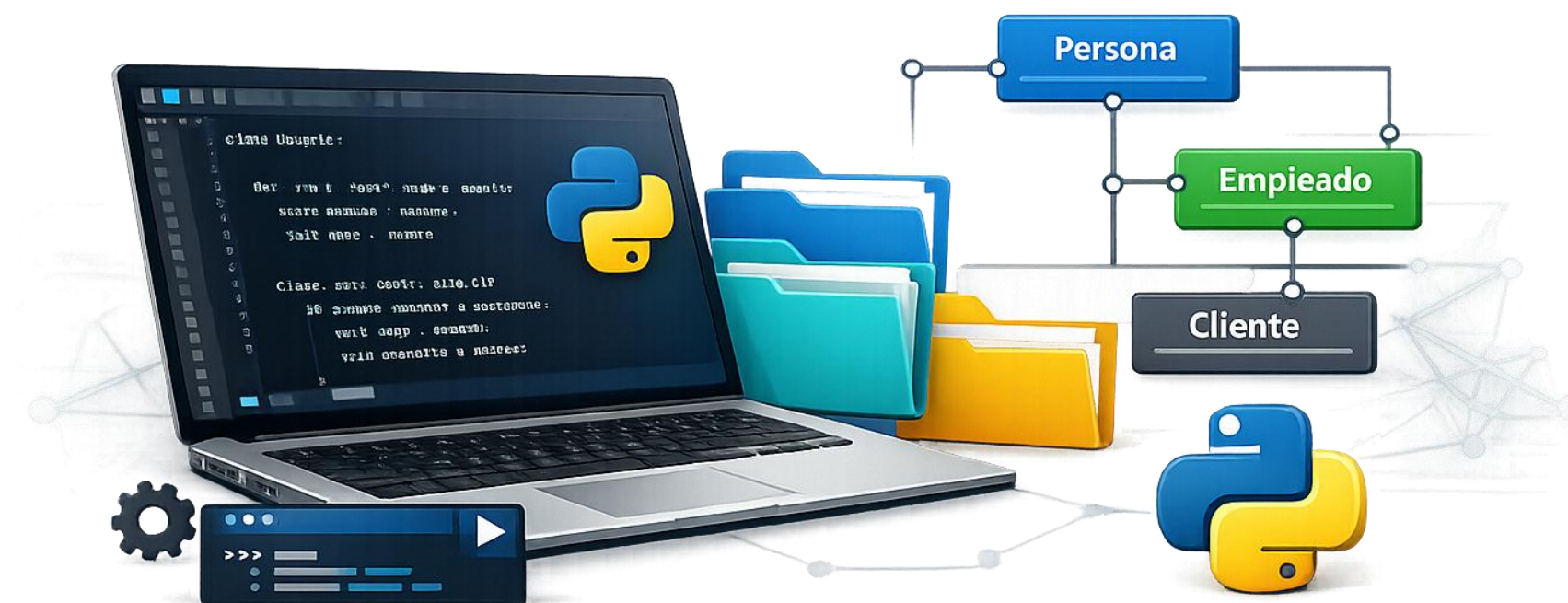
UEA
UNIVERSIDAD
ESTATAL AMAZÓNICA

UNIDAD 1

Fundamentos de la Programación Orientada a Objetos

Tema:

CARACTERÍSTICAS DE LA PROGRAMACIÓN ORIENTADA A OBJETOS





RESULTADO DE APRENDIZAJE

Interpreta los conceptos fundamentales de la programación orientada a objetos POO para el desarrollo de soluciones computacionales de complejidad media.

Al finalizar la sesión, el estudiante será capaz de:

1

Identificar las principales características de la Programación Orientada a Objetos en código Python.

2

Reconocer la función de las clases, objetos, atributos y métodos dentro de un proyecto.

3

Comprender cómo la modularidad facilita la organización y mantenimiento del software.

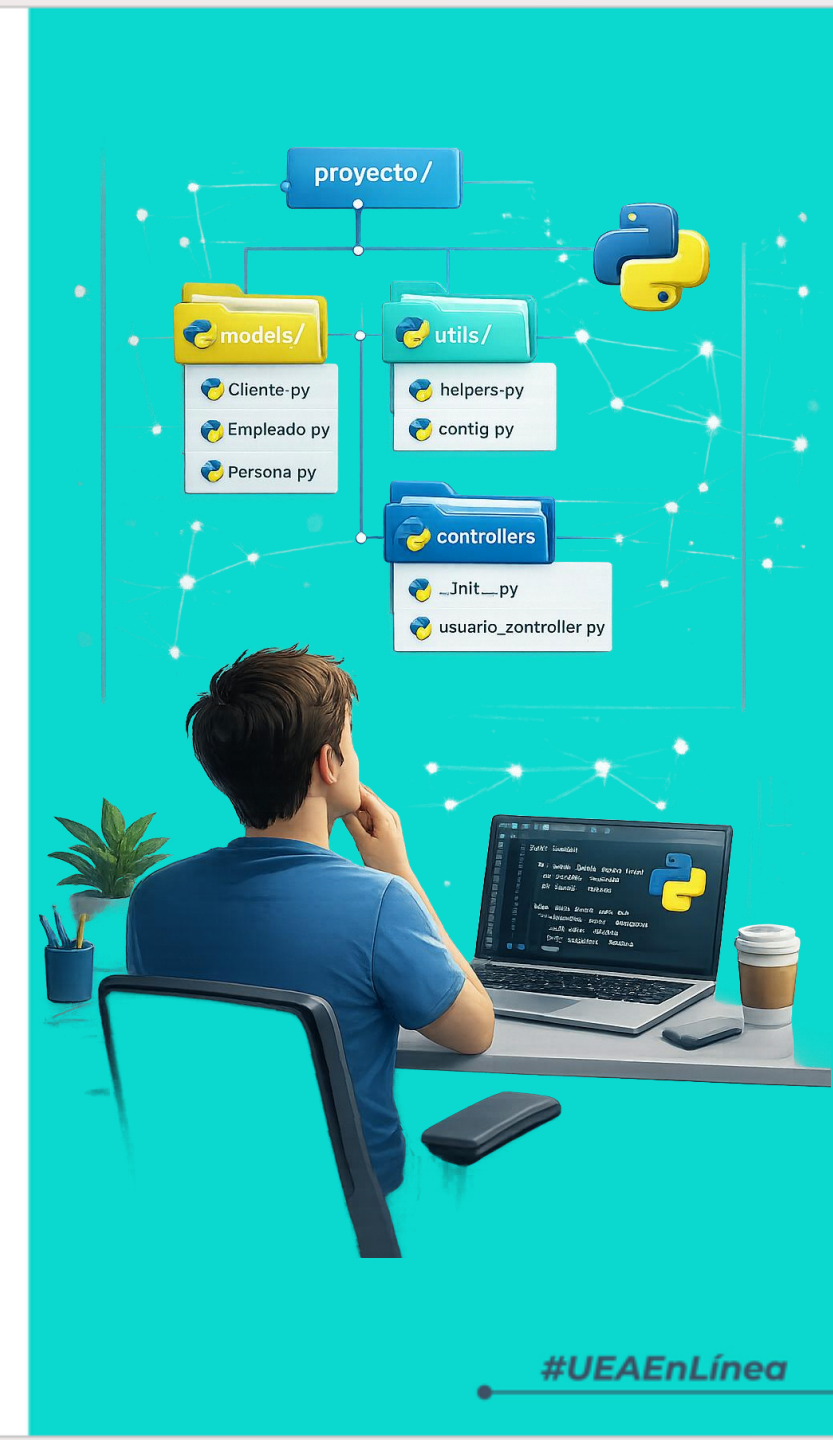
INTRODUCCIÓN

Las características de la POO en proyectos reales

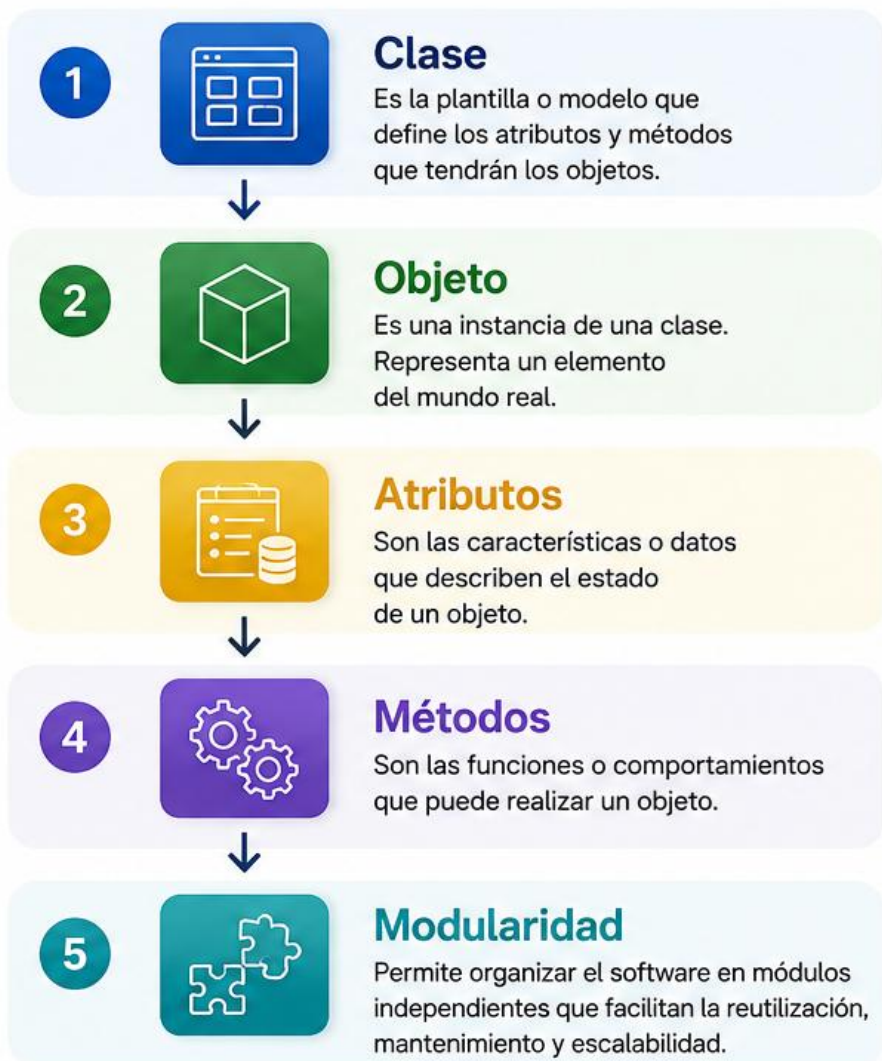
La Programación Orientada a Objetos no solo permite crear clases y objetos. También facilita la organización del software mediante componentes independientes que colaboran entre sí.

Cuando los programas aumentan de tamaño, resulta necesario separar responsabilidades para mejorar la comprensión, reutilización y mantenimiento del código.

En esta sesión analizaremos cómo las características fundamentales de la POO se aplican dentro de un proyecto Python organizado en múltiples archivos.



CARACTERÍSTICAS PRINCIPALES DE LA POO



Ejemplo en Python

```

1  # Clase
2  class Persona:
3      def __init__(self, nombre, edad):
4          self.nombre = nombre  # Atributo
5          self.edad = edad      # Atributo
6
7      def saludar(self):        # Método
8          return f"Hola, soy {self.nombre}"
9
10 # Objeto (instancia de la clase Persona)
11 persona1 = Persona("Ana", 20)
12 persona2 = Persona("Luis", 25)
13
14 print(persona1.saludar())
15 print(persona2.saludar())
  
```

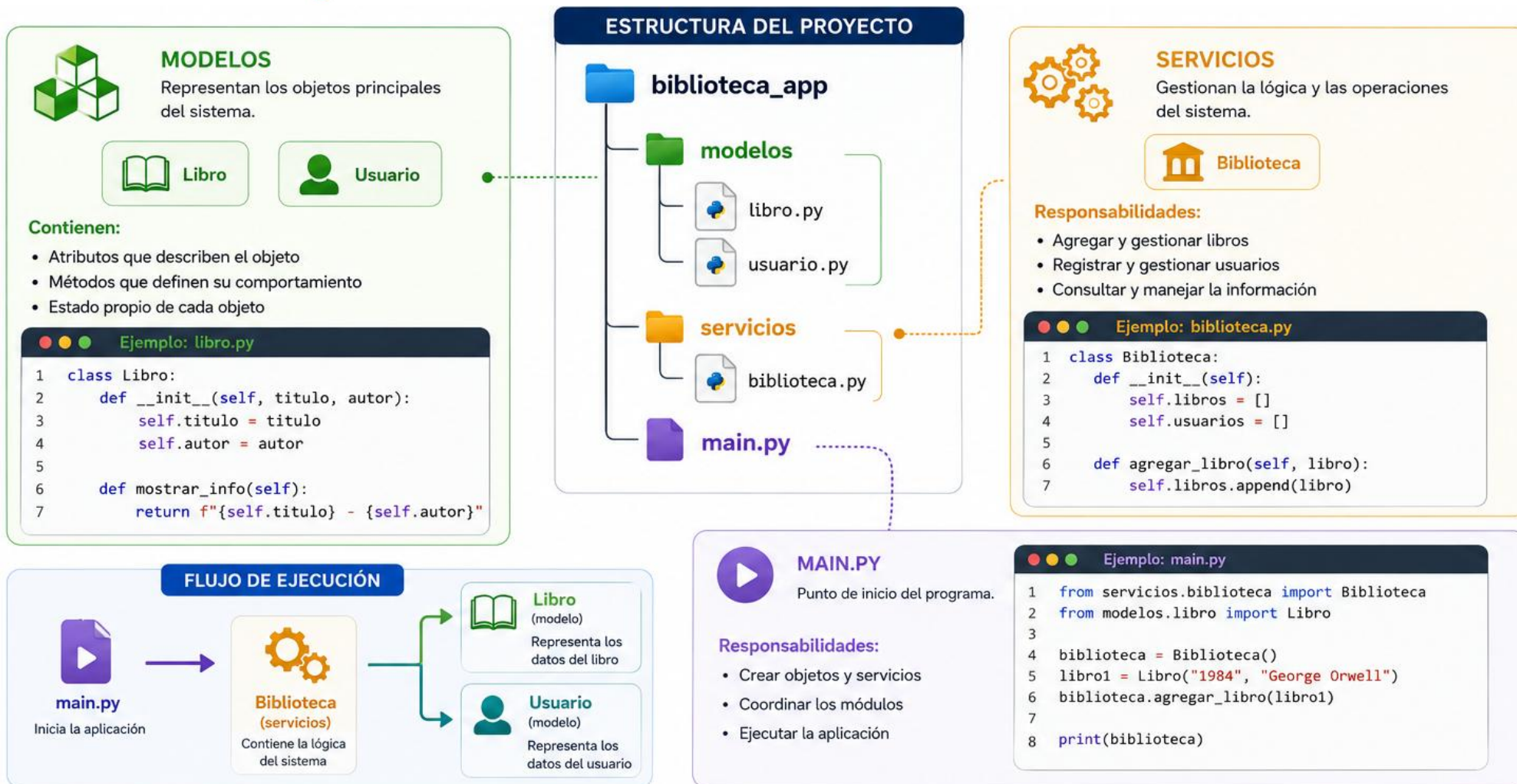
- Clase**: Plantilla o modelo
- Atributos**: Datos que describen el objeto
- Métodos**: Comportamientos del objeto
- Objetos**: Instancias de la clase
- Modularidad**: Organización en módulos reutilizables



En conjunto, estas características hacen posible construir programas más organizados, flexibles y fáciles de mantener.



ORGANIZACIÓN MODULAR DEL PROYECTO

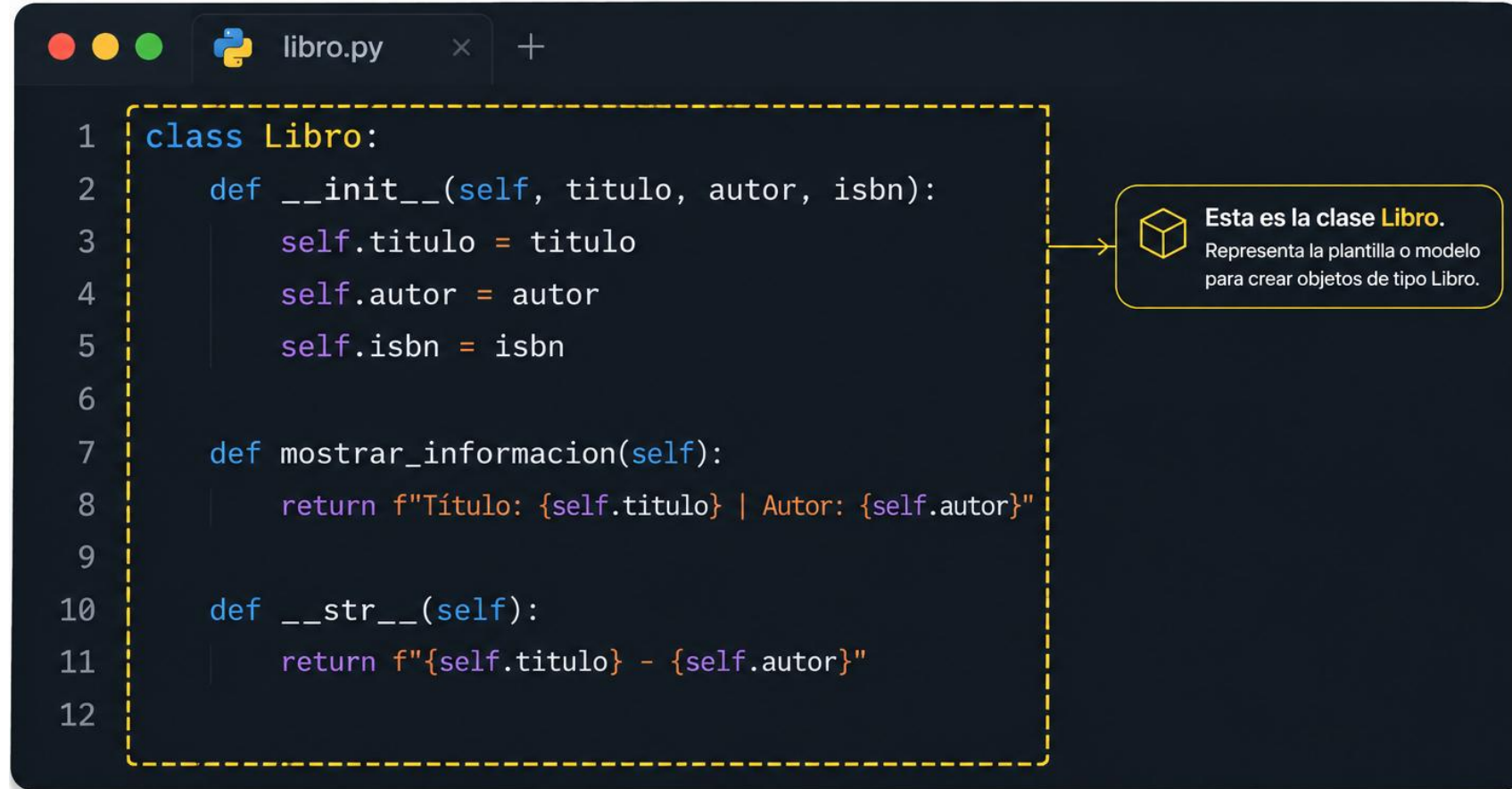


La separación de responsabilidades permite crear aplicaciones más **organizadas, reutilizables y fáciles de mantener.**



LA CLASE

- ❑ La clase representa una entidad del problema que deseamos modelar.
- ❑ En este ejemplo, la clase Libro permite representar información relacionada con libros dentro del sistema.
- ❑ Cada objeto creado a partir de esta clase tendrá sus propios datos y comportamientos.



```
1 class Libro:
2     def __init__(self, titulo, autor, isbn):
3         self.titulo = titulo
4         self.autor = autor
5         self.isbn = isbn
6
7     def mostrar_informacion(self):
8         return f"Título: {self.titulo} | Autor: {self.autor}"
9
10    def __str__(self):
11        return f"{self.titulo} - {self.autor}"
12
```

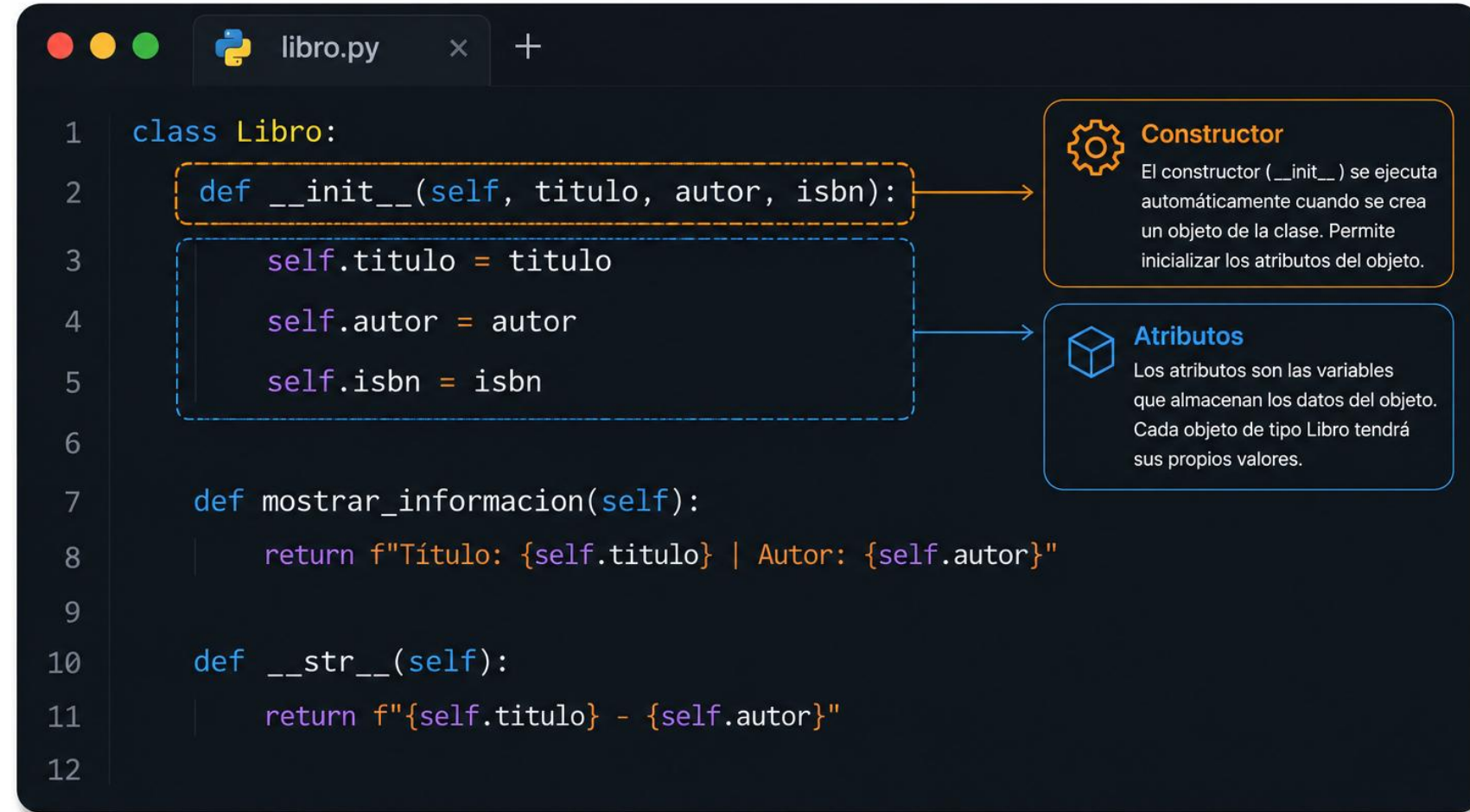
Esta es la clase **Libro**.
Representa la plantilla o modelo para crear objetos de tipo Libro.

CONSTRUCTOR Y ATRIBUTOS

El constructor permite inicializar los atributos del objeto en el momento de su creación.

Los atributos almacenan la información propia de cada objeto. En este caso:

- ☐ titulo
- ☐ autor
- ☐ isbn



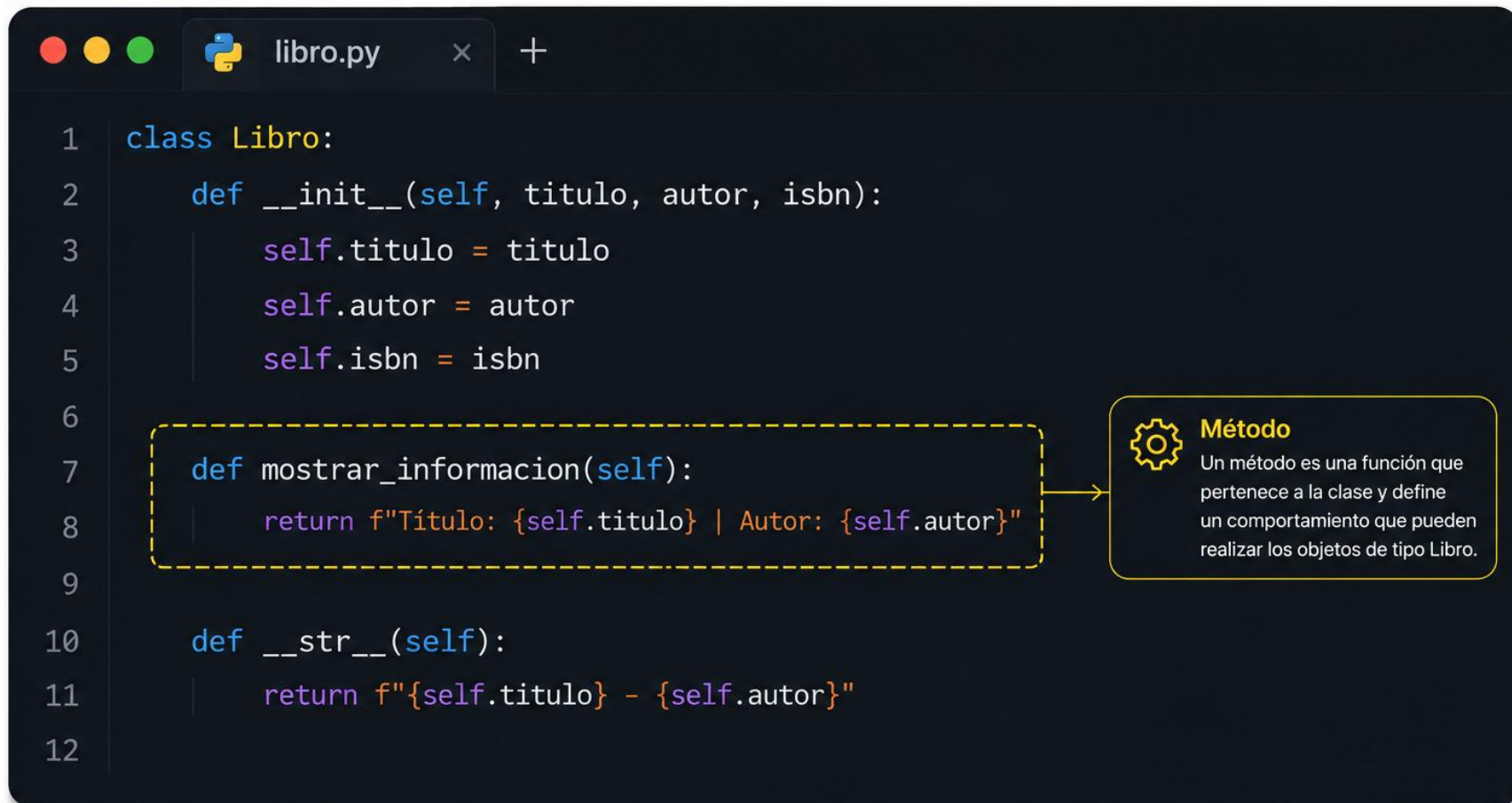
```
1 class Libro:
2     def __init__(self, titulo, autor, isbn):
3         self.titulo = titulo
4         self.autor = autor
5         self.isbn = isbn
6
7     def mostrar_informacion(self):
8         return f"Título: {self.titulo} | Autor: {self.autor}"
9
10    def __str__(self):
11        return f"{self.titulo} - {self.autor}"
12
```

Constructor
El constructor (`__init__`) se ejecuta automáticamente cuando se crea un objeto de la clase. Permite inicializar los atributos del objeto.

Atributos
Los atributos son las variables que almacenan los datos del objeto. Cada objeto de tipo `Libro` tendrá sus propios valores.

MÉTODOS

- ❑ Los métodos representan las acciones que puede realizar un objeto.
- ❑ A diferencia de los atributos, que almacenan información, los métodos permiten ejecutar comportamientos asociados al objeto.
- ❑ En la POO los datos y las acciones permanecen agrupados dentro de una misma estructura.



```
1 class Libro:
2     def __init__(self, titulo, autor, isbn):
3         self.titulo = titulo
4         self.autor = autor
5         self.isbn = isbn
6
7     def mostrar_informacion(self):
8         return f"Título: {self.titulo} | Autor: {self.autor}"
9
10    def __str__(self):
11        return f"{self.titulo} - {self.autor}"
12
```

Método
Un método es una función que pertenece a la clase y define un comportamiento que pueden realizar los objetos de tipo Libro.

MÉTODO ESPECIAL STR()

- ❑ Python incorpora métodos especiales que permiten adaptar el comportamiento de los objetos.
- ❑ El método `__str__()` define cómo se mostrará un objeto cuando se utilice la función `print()`.
- ❑ Esto mejora la legibilidad y facilita la interpretación de la información durante la ejecución del programa.

```
libro.py x +
1 class Libro:
2     def __init__(self, titulo, autor, isbn):
3         self.titulo = titulo
4         self.autor = autor
5         self.isbn = isbn
6
7     def mostrar_informacion(self):
8         return f"Título: {self.titulo} | Autor: {self.autor}"
9
10    def __str__(self):
11        return f"{self.titulo} - {self.autor}"
12
```

 **Método Especial `__str__()`**
Permite definir cómo se representa un objeto de la clase cuando se convierte a cadena de texto. Se utiliza, por ejemplo, al imprimir el objeto con `print()`.

RELACIÓN ENTRE OBJETOS

- Una de las principales ventajas de la POO es que los objetos pueden interactuar entre sí.

```
1  from modelos.libro import Libro
2  from modelos.usuario import Usuario
3  from servicios.biblioteca import Biblioteca
4
5  # Crear objetos Libro
6  libro1 = Libro(
7      "Python para Principiantes",
8      "Juan Pérez",
9      "ISBN001"
10 )
11
12 libro2 = Libro(
13     "Programación Orientada a Objetos",
14     "María Gómez",
15     "ISBN002"
16 )
17
18 # Crear objeto Usuario
19 usuario1 = Usuario(
20     "Carlos López",
21     "carlos@correo.com"
22 )
23
24 # Crear biblioteca
25 biblioteca = Biblioteca()
26
27 # Agregar objetos a la biblioteca
28 biblioteca.agregar_libro(libro1)
29 biblioteca.agregar_libro(libro2)
30 biblioteca.agregar_usuario(usuario1)
```

★ LÍNEA CLAVE

```
biblioteca.agregar_libro(libro1)
```

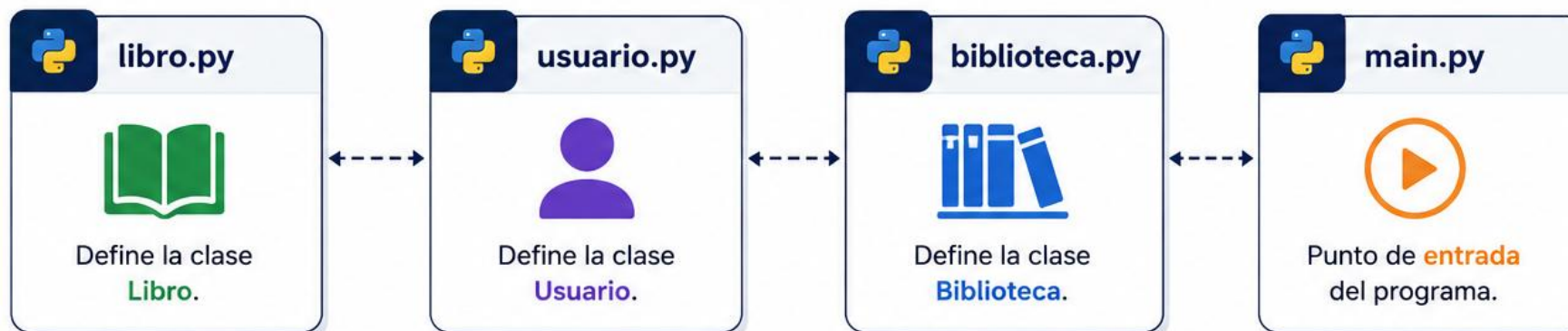
Con esta línea, el objeto **biblioteca** (de la clase **Biblioteca**) recibe el objeto **libro1** (de la clase **Libro**) y lo administra dentro del sistema.

RELACIÓN ENTRE OBJETOS



MODULARIDAD

La **modularidad** consiste en dividir un programa en **componentes independientes**.



Cada módulo contiene una **responsabilidad específica**, permitiendo:



REUTILIZACIÓN

Los módulos pueden ser usados en diferentes partes del programa o en otros proyectos.



ORGANIZACIÓN

El código se estructura de forma lógica y ordenada, facilitando su comprensión.



FACILIDAD DE MANTENIMIENTO

Al estar dividido, es más fácil localizar, corregir y actualizar partes del código sin afectar el resto.



ESCALABILIDAD

El sistema puede crecer incorporando nuevos módulos sin alterar los existentes.



Esta característica es ampliamente utilizada en el **desarrollo profesional de software**, ya que permite construir aplicaciones más **robustas, flexibles y eficientes**.

USO DE IMPORTACIONES

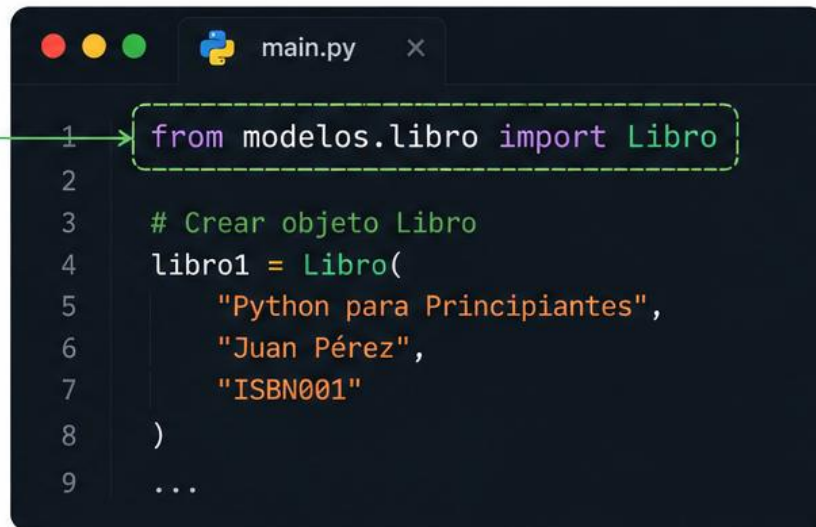


Las importaciones permiten **reutilizar** clases y funciones definidas en otros archivos.
Gracias a este mecanismo, los módulos pueden **comunicarse** entre sí sin necesidad de duplicar código.

CÓDIGO

```
1 from modelos.libro import Libro
```

ESTRUCTURA DEL PROYECTO



main.py importa la clase **Libro** desde el módulo **modelos.libro** para poder crear objetos de esa clase.

¿CÓMO FUNCIONA?



main.py

(Archivo principal)



Importa

la clase **Libro**



Libro

(Desde **modelos/libro.py**)



Esto permite que **main.py** use la clase **Libro** sin tener que escribir su código nuevamente, promoviendo la **reutilización** y la **organización** del proyecto.



USO DE IMPORTACIONES



Organizar el proyecto en **módulos y clases separadas** no es solo una buena práctica, sino que aporta **beneficios concretos** en el desarrollo real de software.



CÓDIGO DESORGANIZADO

 programa.py

Clases, funciones y lógica en un solo archivo

class Libro:

...

class Usuario:

...

class Biblioteca:

...

def agregar_libro():

...

def mostrar_menu():

...

Lógica del programa

...



DIFÍCIL MANTENIMIENTO

Todo el código está junto, por lo que cualquier cambio puede afectar otras partes del programa.



REPETICIÓN

Es común duplicar código porque no hay una estructura clara que permita reutilizarlo.



CONFUSIÓN

A medida que el proyecto crece, resulta difícil entender el flujo y encontrar lo que se necesita.



POCO ESCALABLE

Agregar nuevas funcionalidades se vuelve complicado y desordenado.

VS



CÓDIGO MODULAR (COMO EN NUESTRO PROYECTO)

mi_proyecto/

modelos/

libro.py

usuario.py

servicios/

biblioteca.py

main.py



CLARIDAD

Cada módulo tiene una responsabilidad específica, lo que hace el código más legible y comprensible.



REUTILIZACIÓN

Las clases (Libro, Usuario, Biblioteca) pueden usarse en otros proyectos o módulos sin modificar su código.



ESCALABILIDAD

Es fácil agregar nuevas funcionalidades o módulos sin afectar lo existente.



MANTENIMIENTO SENCILLO

Los cambios se realizan en módulos específicos, reduciendo el riesgo de errores y facilitando las actualizaciones.



Esta organización basada en la Programación Orientada a Objetos (POO) permite construir aplicaciones más **robustas, flexibles y profesionalmente estructuradas**.

REFLEXIÓN

Comprender las diferencias entre los paradigmas de programación permite seleccionar la estrategia más adecuada para cada proyecto.

Si tuvieras que desarrollar un sistema completo para una universidad, ¿cómo organizarías el software?





Conclusiones

- La Programación Orientada a Objetos permite representar elementos del mundo real mediante clases y objetos, facilitando el diseño de soluciones más organizadas y comprensibles.
- Los atributos y métodos permiten agrupar datos y comportamientos dentro de una misma estructura, favoreciendo una mejor organización del código.
- La combinación de clases, objetos, módulos e importaciones permite desarrollar aplicaciones más estructuradas y preparadas para crecer conforme aumentan los requerimientos del sistema.
- La modularidad posibilita dividir un proyecto en componentes con responsabilidades específicas, mejorando la reutilización, el mantenimiento y la escalabilidad del software.

Hernández Bejarano, M., & Baquero Rey, L. E. (2023). Programación orientada a objetos en Java: Buenas prácticas. Ediciones de la U.

Hernández Bejarano, M., & Baquero Rey, L. E. (2025). Python con orientación a objetos y al análisis de datos. Ediciones de la U.

Bibliografía

Boucheny, V. (2025). Aprender programación orientada a objetos con Python (3.^a ed.).

Ediciones ENI. Piñeiro Gómez, J. M. (2022). Principios de la programación orientada a objetos. Paraninfo.



UEA
UNIVERSIDAD
ESTATAL AMAZÓNICA

JUNTOS TRANSFORMAMOS EL MUNDO DESDE LA AMAZONÍA



**CARRERAS
EN LÍNEA**

***Dr. David Sancho, PhD.
RECTOR UEA***